education

Деревья поиска

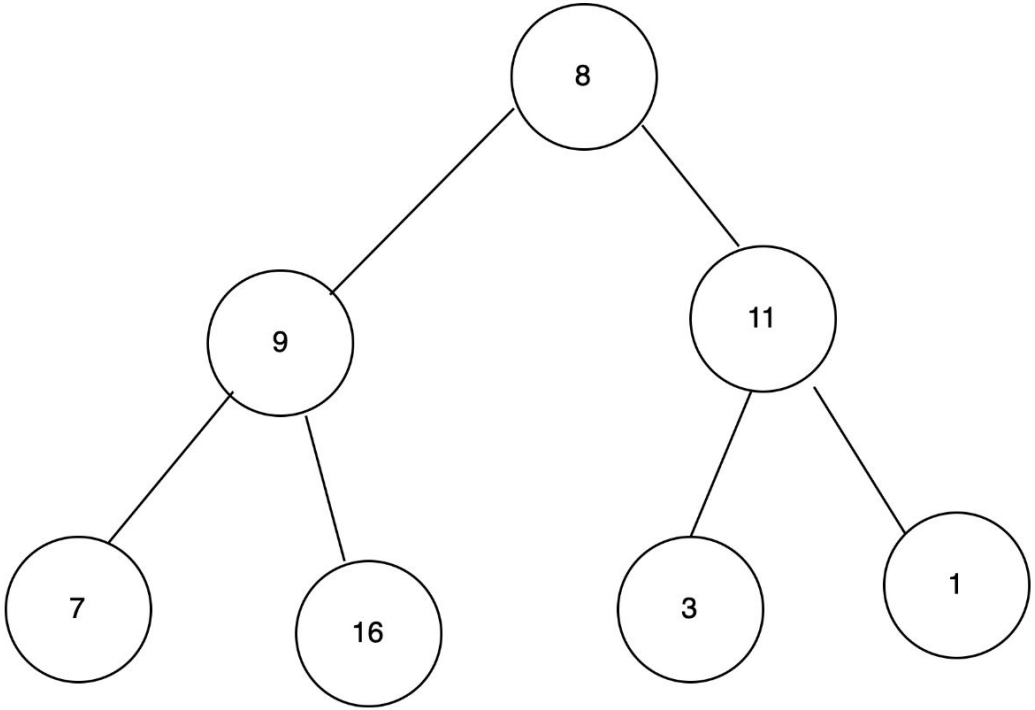
Семинар

О чём поговорим?

- Восстановление бинарного дерева из массива
- Проверка на симметричность бинарного дерева
- Поиск минимальной глубины
- Произведение минимального и максимального значения
- Сравнение двух деревьев
- Вид бинарного дерева справа

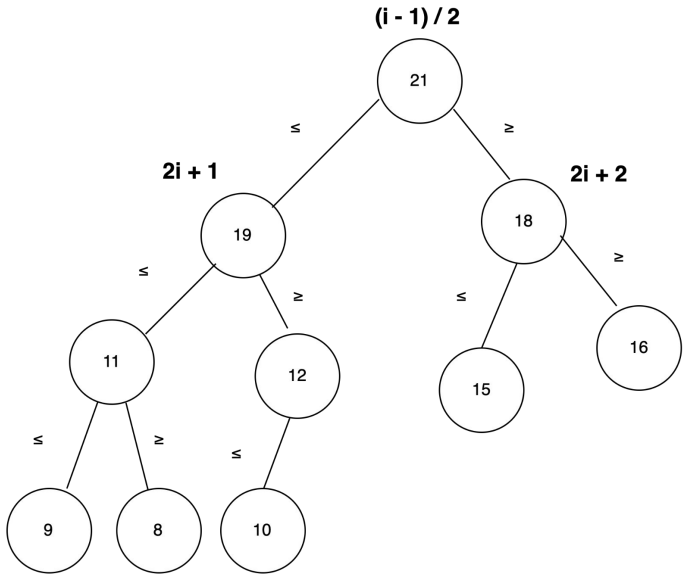
Восстановление бинарного дерева из массива

Представление бин дерева массиве



Представление бин дерева массиве

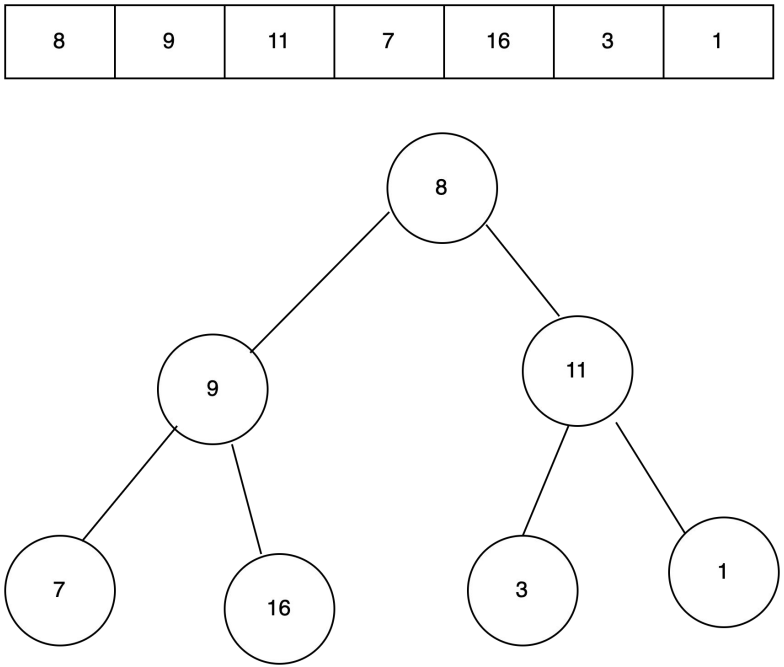
- Корень: всегда находится под нулевым индексом
- Родительский узел: **(i - 1) / 2**
- Левый потомок: **2i + 1**
- Для узла 12 (индекс 4) его левый потомок, если он есть, то будет находится под индексом
  2*4 + 1 = 9
- Правый потомок: **2i + 2**
- Для узла 11 (индекс 3) его правый потомок, если он есть,
  то будет находиться под индексом 2*3 + 2 = 8



21	19	18	11	12	15	16	9	8	10

Восстановление бинарного дерева из массива

Необходимо реализовать функцию, которая будет принимать на вход массив и выстраивать из него бинарное дерево.



Восстановление бинарного дерева из массива

```
class TreeNode{
    function TreeNode(val = 0, left = null, right = null) {
        this.data = val
        this.left = left
        this.right = right
    }
}

function buildTree(arr, i) {
    // определяем базовые кейсы
    // создаем узел
    // добавляем левого потомка
    // добавляем правого потомка

    return root
}
```

Восстановление бинарного дерева из массива

- Необходимо рекурсивно для каждого узла достраивать бинарное дерево
- Строить дерево начинаем с корня, то есть с нулевого индекса

```
function buildTree(arr, i) {
    if i >= len(arr) {
        return null
    }

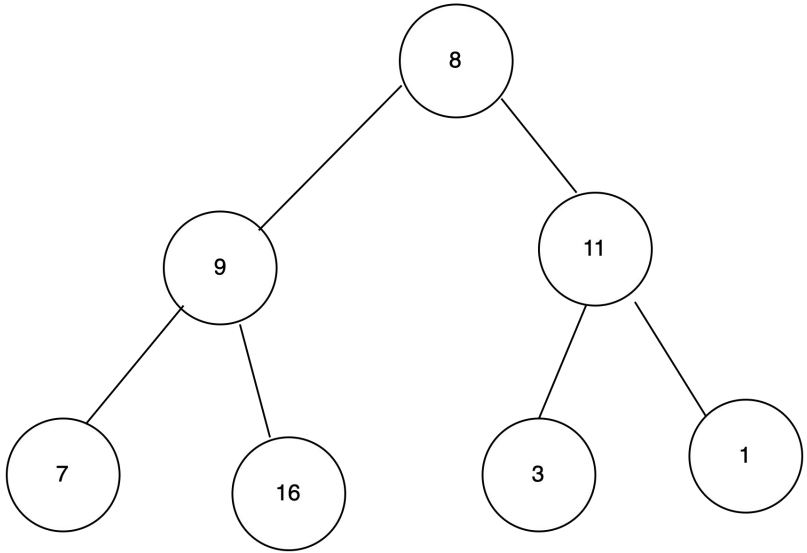
    root = TreeNode(arr[i])
    // добавляем левого потомка
    // добавляем правого потомка

    return root
}
```

Восстановление бинарного дерева из массива

- Рекурсивно вызываем buildTree для каждого узла

8	9	11	7	16	3	1

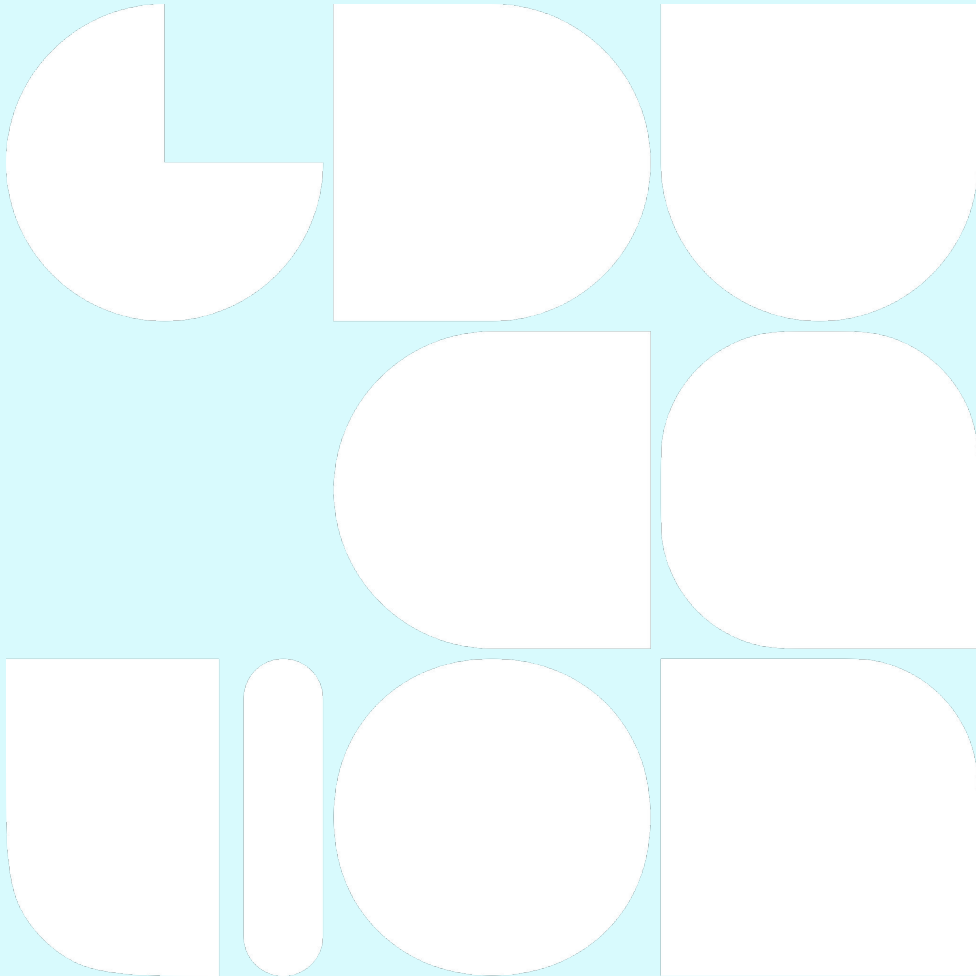



```
function buildTree(arr, i) {
    if i >= len(arr) {
        return null
    }

    root = TreeNode(arr[i])
    root.left = buildTree(arr, 2 * i + 1)
    root.right = buildTree(arr, 2 * i + 2)

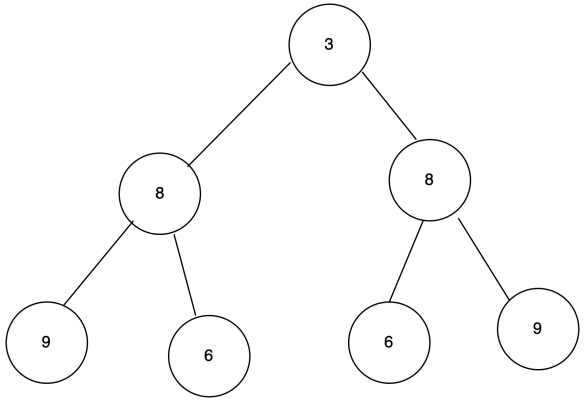
    return root
}
```

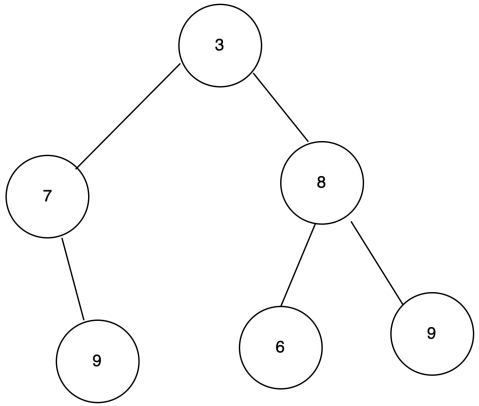
Симметричное бинарное дерево



Симметричное бинарное дерево

- На вход функции подается бинарное дерево.
- Необходимо понять, является ли это дерево симметричным.



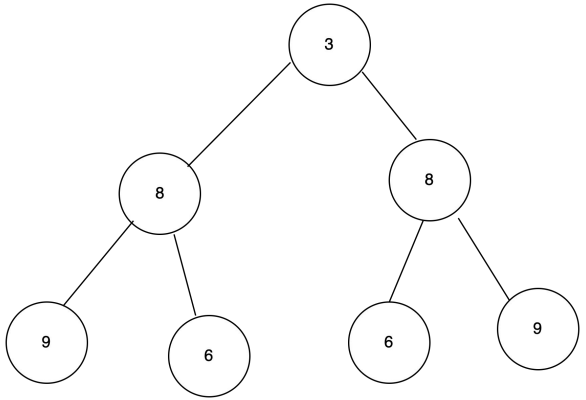


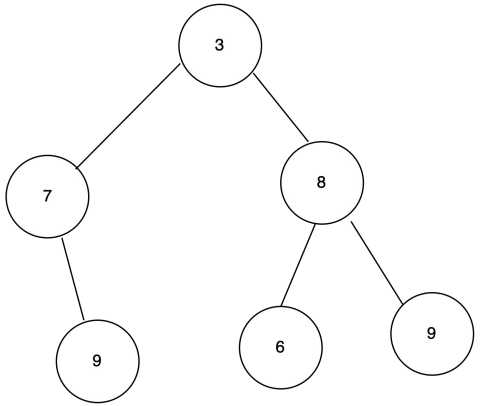
Дерево сверху — симметричное.

Дерево снизу — нет.

Симметричное дерево

- Можно решить двумя способами: итеративно или рекурсивно





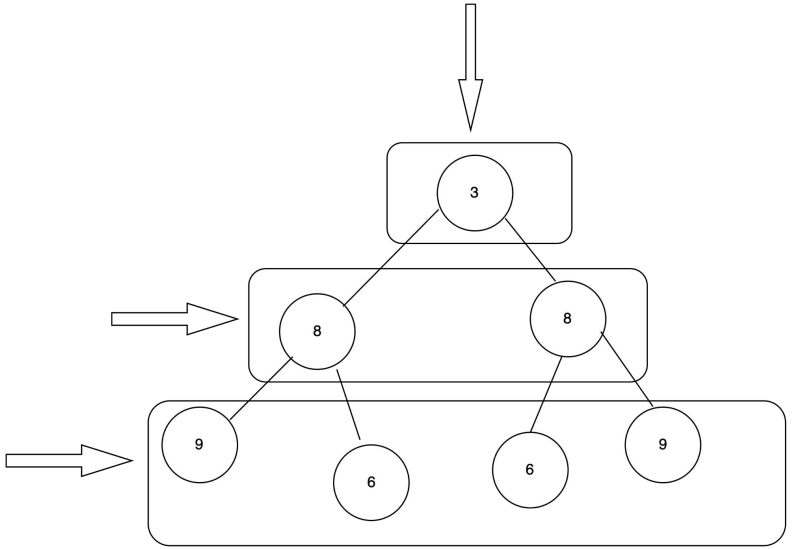
Симметричное бинарное дерево

Breadth-First Search

Обход бинарного дерева в ширину

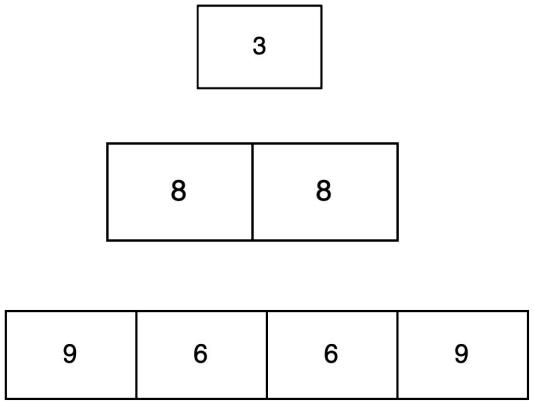
Симметричное дерево BFS (Breadth-First Search)

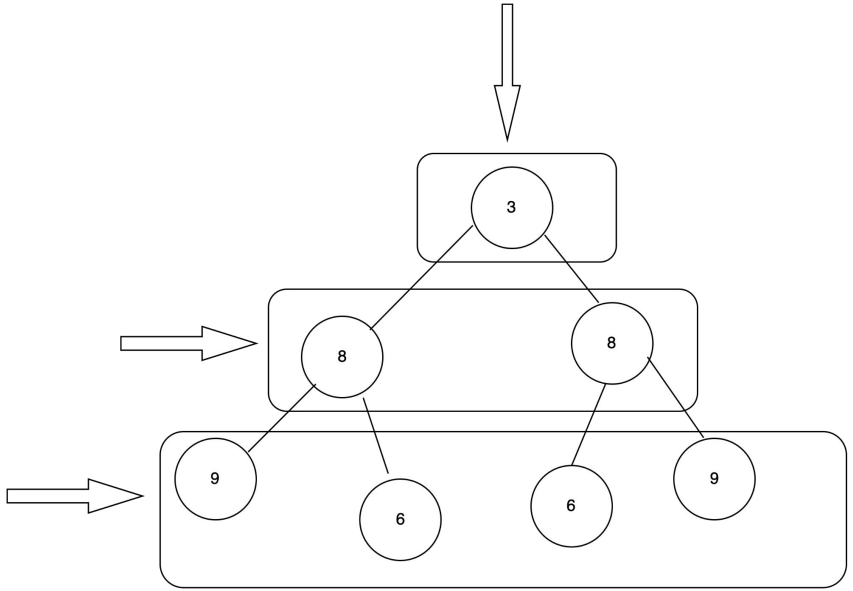
Сверху вниз, слева направо, проходим последовательно все уровни.



Симметричное дерево BFS

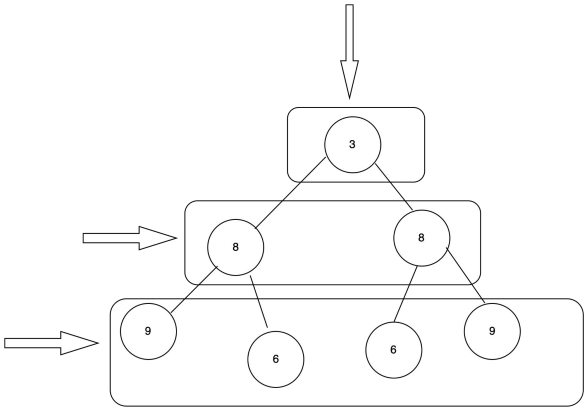
- На каждой итерации решаем задачу, является ли уровень палиндромом





Симметричное дерево BFS

- Создаем массив узлов дерева, чтобы можно было по нему итерироваться
- На первой итерации в массиве только корень
- Для каждого уровня создаем очередь из узлов, в которую будем складывать всех детей текущего уровня. То есть для 2 уровня (8 и 8) в очереди будут храниться узлы 9, 6, 6, 9.

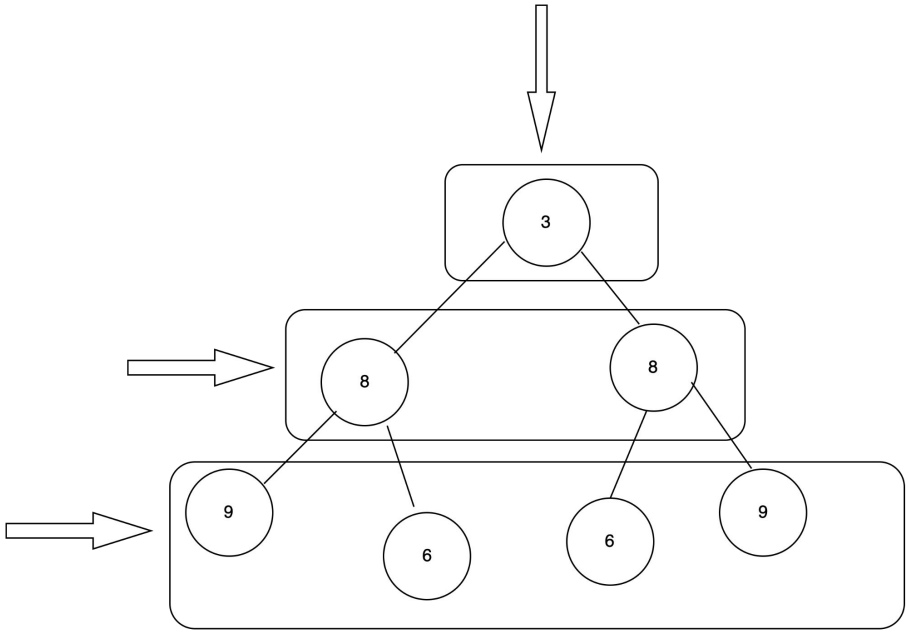



```
function isSymmetric(root) {
    if root == null {
        return true;
    }
    queue = [root];

    while (len(queue) > 0) {
        ...
    }

    return true;
}
```

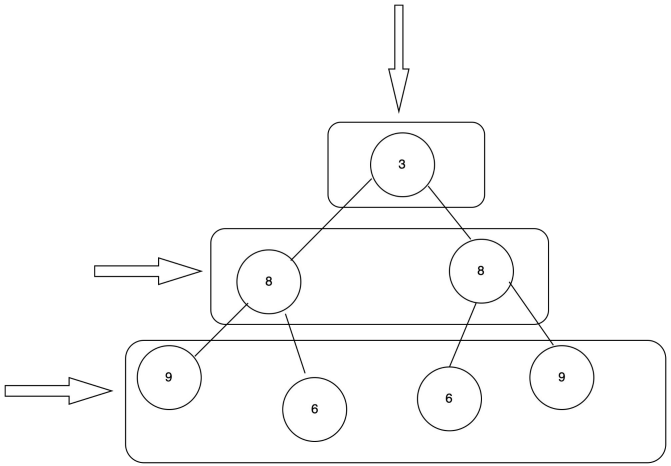
Симметричное дерево BFS




```
function isSymmetric(root) {
    if root == null {
        return true;
    }
    queue = [root];

    while (len(queue) > 0) {
        queueLen = len(queue);
        for (i = 0; i < queueLen; i++) {
            ...
        }
        queue = queue[queueLen:];
    }

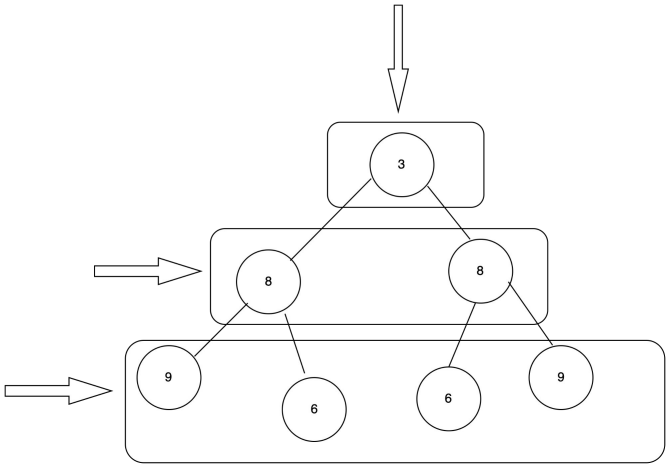
    return true;
}
```




```
function isSymmetric(root) {
    if root == null {
        return true;
    }
    queue = [root];

    while (len(queue) > 0) {
        queueLen = len(queue);
        for (i = 0; i < queueLen; i++) {
            //queue[queueLen - i - 1] - симметричный
            //относительно узла queue[i]
            if (queue[i] == null && queue[queueLen - i - 1] == null) {
                continue;
            }
            ???
        }
        queue = queue[queueLen:];
    }

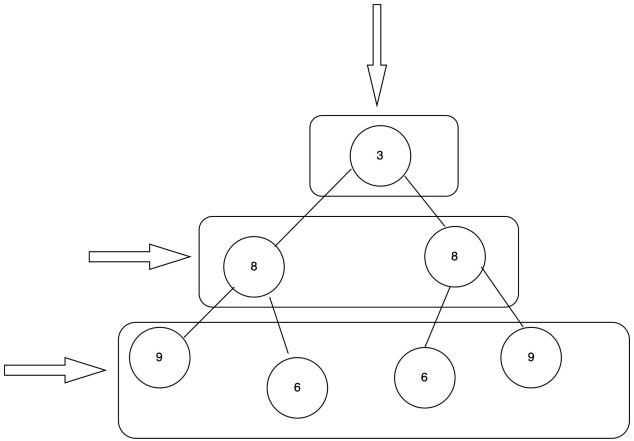
    return true;
}
```




```
function isSymmetric(root) {
    if root == null {
        return true;
    }
    queue = [root];

    while (len(queue) > 0) {
        queueLen = len(queue);
        for (i = 0; i < queueLen; i++) {
            //queue[queueLen - i - 1] - симметричный
           //относительно узла queue[i]
            if (queue[i] == null && queue[queueLen - i - 1] == null) {
                continue;
            }
            if (queue[i] == null || queue[queueLen - i - 1] == null) {
                return false;
            }
             ???
        }
        queue = queue[queueLen:];
    }

    return true;
}
```




```
function isSymmetric(root) {
    if root == null {
        return true;
    }
    queue = [root];

    while (len(queue) > 0) {
        queueLen = len(queue);
        for (i = 0; i < queueLen; i++) {
            //queue[queueLen - i - 1] - симметричный
           //относительно узла queue[i]
            if (queue[i] == null && queue[queueLen - i - 1] == null) {
                continue;
            }
            if (queue[i] == null || queue[queueLen - i - 1] == null) {
                return false;
            }
            if (queue[i].data != queue[queueLen - i - 1].data) {
                return false;
            }

            queue.append(queue[i].left);
            queue.append(queue[i].right);
        }
        queue = queue[queueLen:];
    }

    return true;
}
```

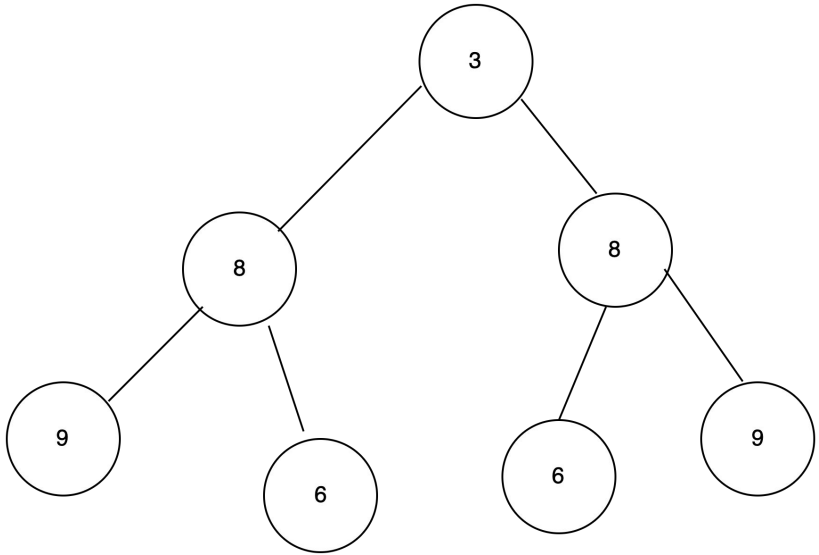
Симметричное дерево

Depth-First Search

Обход дерева в глубину

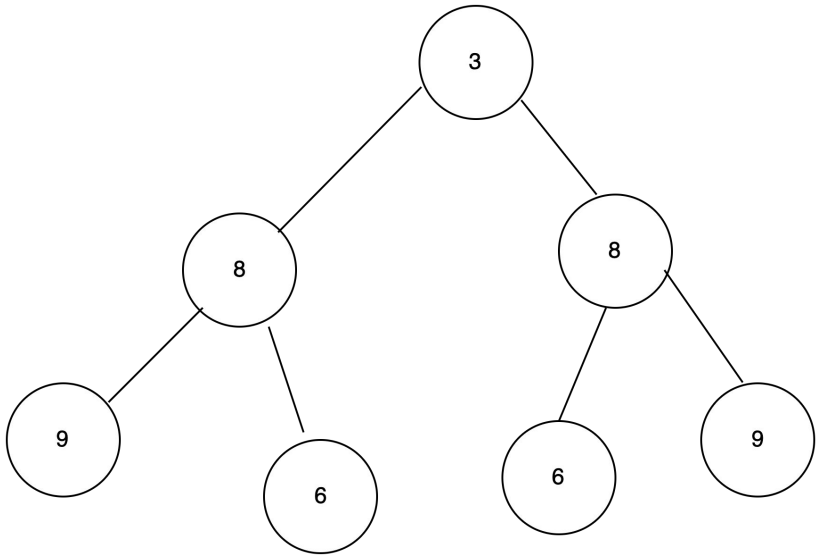
Симметричное дерево
DFS (Depth-First Search)

- Какой вариант обхода DFS подойдет в данном случае?
- LNR RNL NLR ...

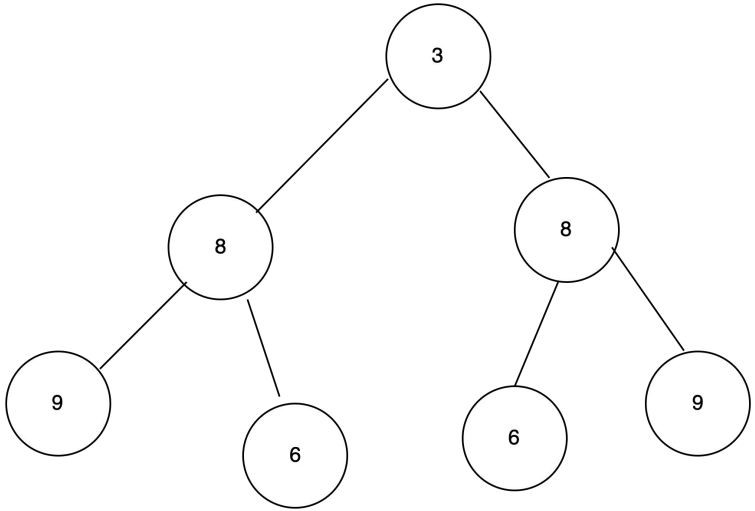


Симметричное дерево DFS

- Самый простой вариант — это вариант, при котором при прохождении по бинарному дереву поиска получили бы отсортированный массив **LNR**.
- **[9, 8, 6, 3, 6, 8, 9]**



Симметричное дерево DFS



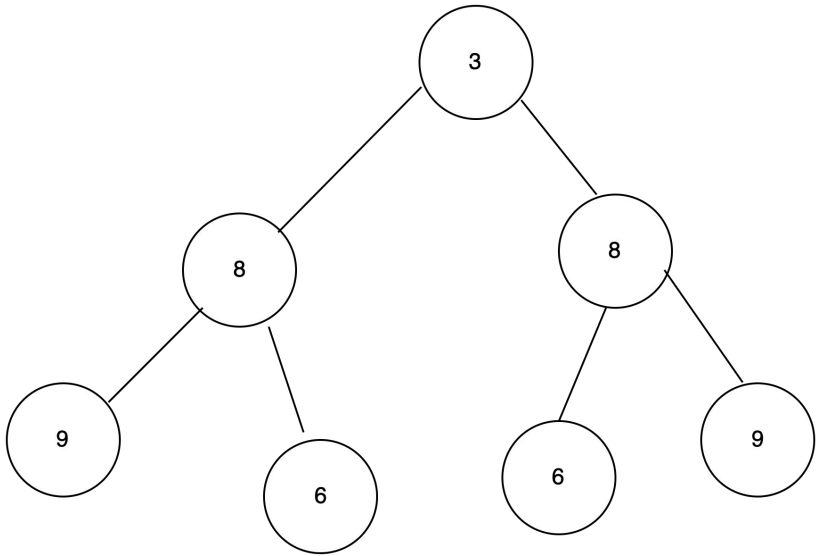

```
function deptSearch(root, res) {
    if root == null {
        return res
    }

    deptSearch(root.left, res)
    res.append(root.data)
    deptSearch(root.right, res)

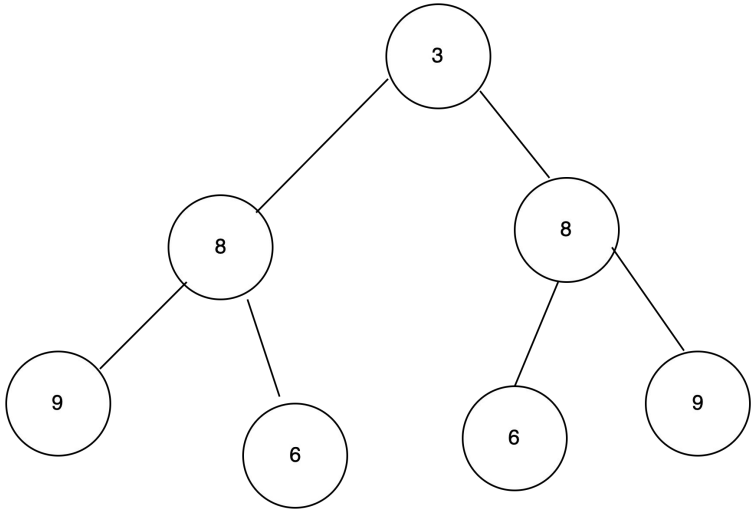
    return res
}
```

Симметричное дерево DFS

- Если корень пустой, считаем, что дерево симметрично
- Обходим дерево сохраняя в массив результат обхода
- Как раньше проверяли очередь — проверяем массив на симметричность



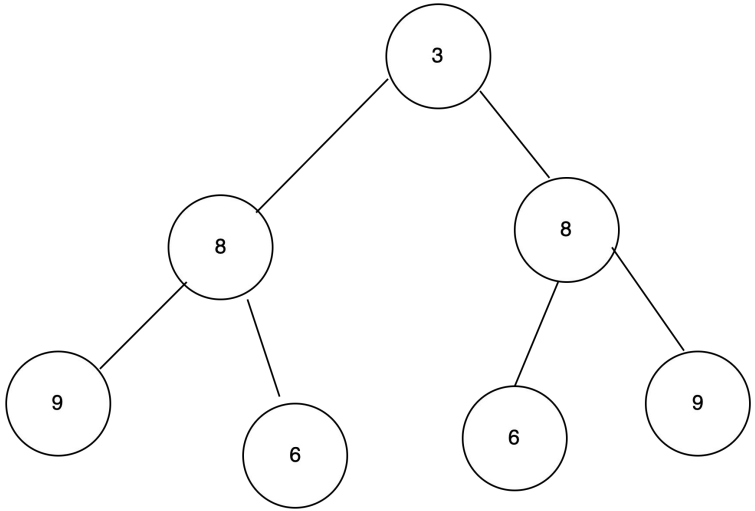
Симметричное дерево DFS




```
function isSymmetricDFS(root) {
    if root == null {
        return true
    }
    data = []
    data = deptSearch(root, data)
    // проверка data на симметричность

    return true
}
```

Симметричное дерево DFS



[9, 8, 6, 3, 6, 8, 9]

```
function isSymmetricDFS(root) {
    if root == null {
        return true
    }
    data = []
    data = deptSearch(root, data)
    j = len(data) - 1
    for i = 0; i < len(data)/2; i++ {
        if data[i] != data[j] {
            return false
        }
        j--
    }

    return true
}
```

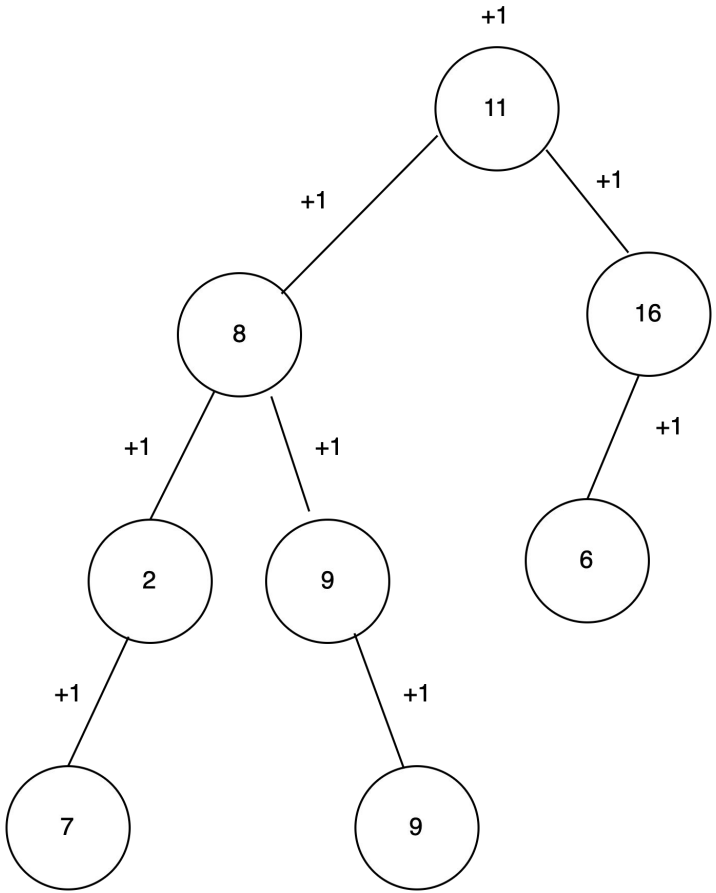
Поиск минимальной глубины бинарного дерева

Поиск минимальной глубины бинарного дерева

На вход функции подается бинарное дерево. Необходимо найти минимальную глубину дерева.

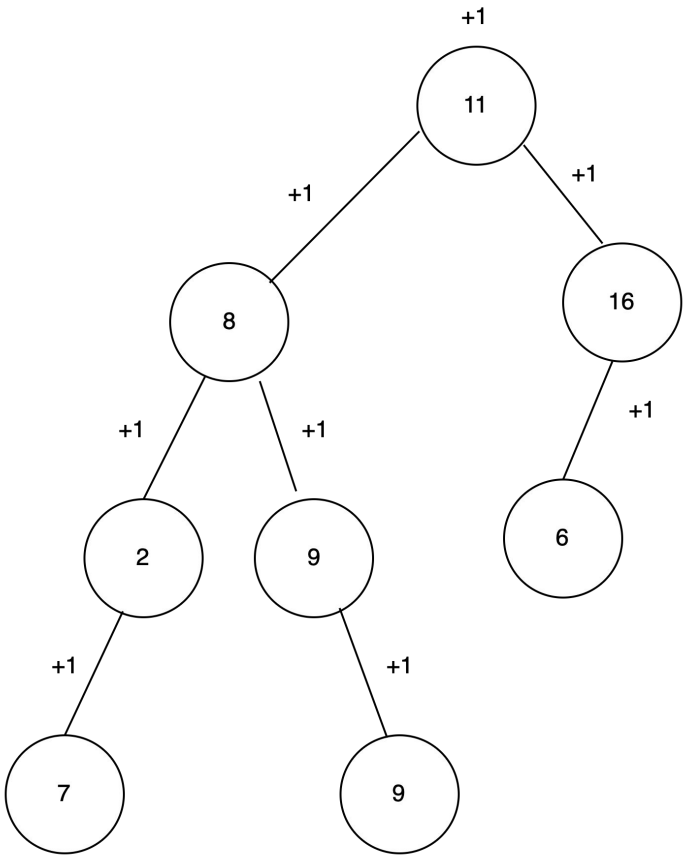
Минимальная глубина — это количество узлов на кратчайшем пути от корневого узла до ближайшего листового узла.

Для данного дерева минимальная глубина 3: узел 11 -> узел 16 -> узел 6

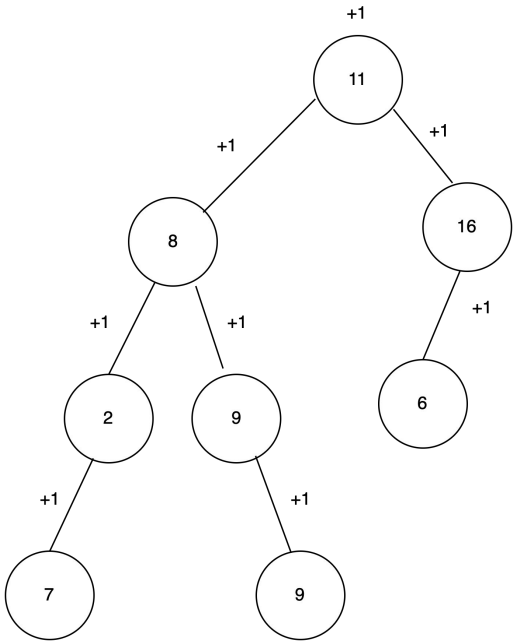


Поиск минимальной глубины бинарного дерева DFS

- Если root не существует, значит, на этом уровне глубина равна нулю
- Если у узла есть и левый, и правый потомки, а значит, есть и левое, и правое поддеревья, необходимо вернуть минимальную глубину из этих поддеревьев
- К результату необходимо прибавить 1, чтобы учесть корневой уровень
- Если есть только левое/правое поддерево, продолжаем поиск только по нему, плюс корневой уровень



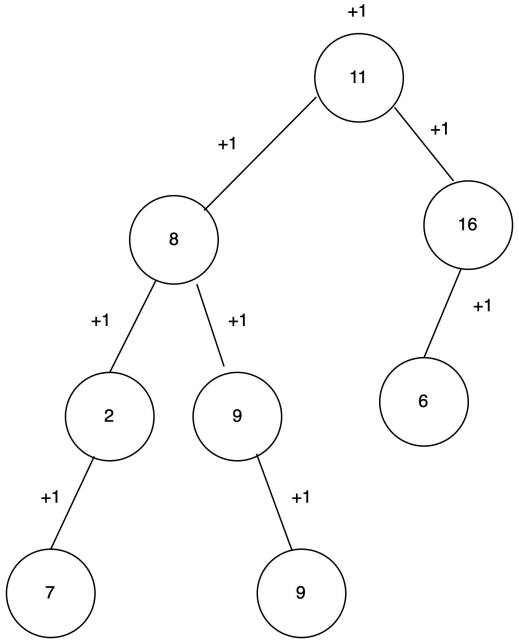
Поиск минимальной глубины бинарного дерева DFS




```
function minDepth(root) {
    if root == null {
        return 0
    }
    if root.left == nil && root.right == nil {
      return 1
    }
    if root.left != null && root.right != null {
        return 1 + min(minDepth(root.left),
                minDepth(root.right))
    }

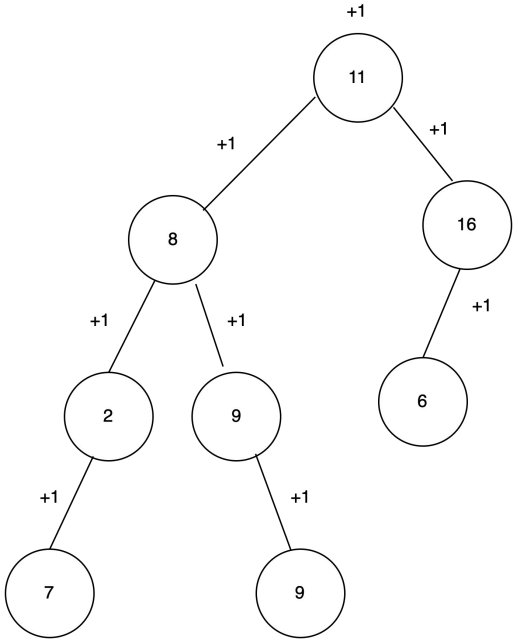
    ...
}
```

Поиск минимальной глубины бинарного дерева DFS




```
function minDepth(root) {
    if root == null {
        return 0
    }
    if root.left == nil && root.right == nil {
        return 1
    }
    if root.left != null && root.right != null {
        return 1 + min(minDepth(root.left),
                minDepth(root.right))
    }
    if root.left != null {
        return 1 + minDepth(root.left)
    }
    ???
}
```

Поиск минимальной глубины бинарного дерева DFS

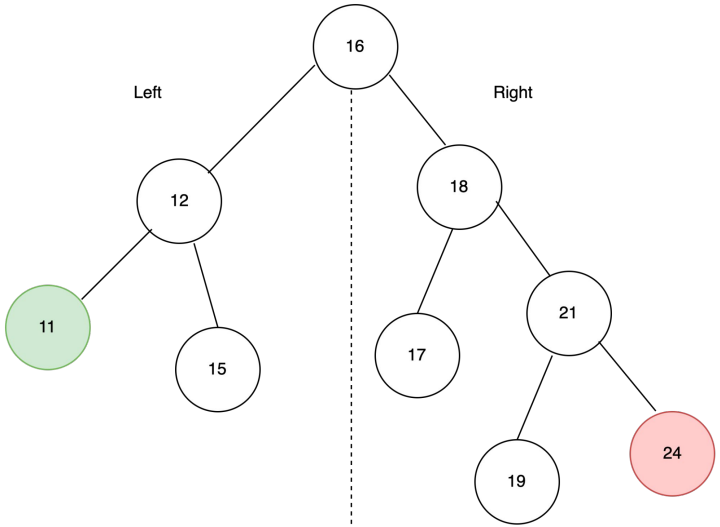



```
function minDepth(root) {
    if root == null {
        return 0
    }
    if root.left == nil && root.right == nil {
      return 1
    }
    if root.left != null && root.right != null {
        return 1 + min(minDepth(root.left),
                minDepth(root.right))
    }
    if root.left != null {
        return 1 + minDepth(root.left)
    }
    if root.right != null {
        return 1 + minDepth(root.right)
    }
}
```

Поиск произведения максимального и минимального элементов

Дано бинарное дерево поиска в виде **массива**. Необходимо найти произведение минимального и максимального значений.

Поиск максимального произведения

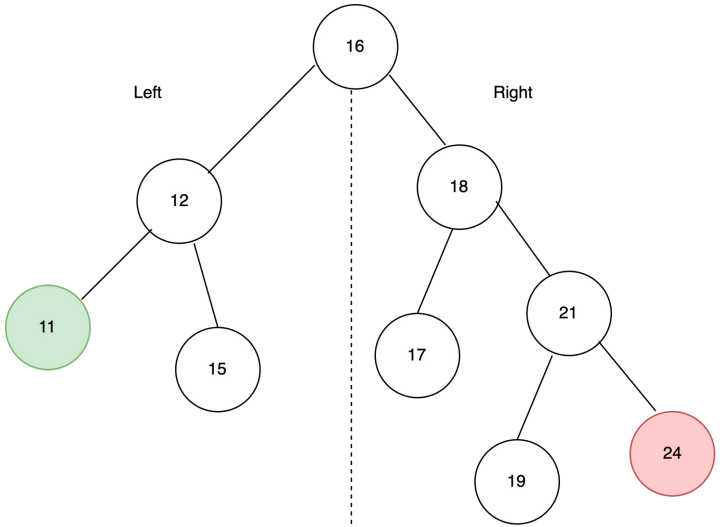



```
function maxMinMultiplication (data) {
    if length(data) == 1 {
        return -1
    }

    // определяем индекс минимального элемента
    // устанавливаем min index равным 1
    // далее в цикле двигаем его на 2 * i + 1
    // определяем индекс максимального
элемента
    // устанавливаем max index равным 2
    // далее двигаем его на 2 * i + 2

    result = data[min_index] * data[max_index]
    return result
}
```

Поиск максимального произведения



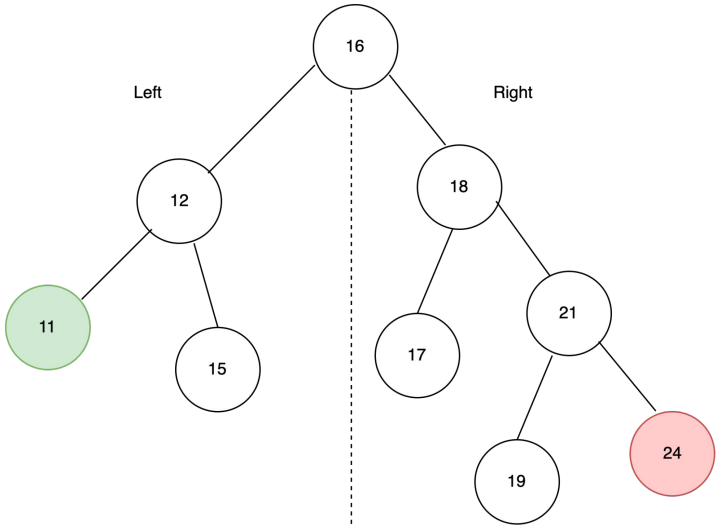

```
function maxMinMultiplication (data) {
    if length(data) < 3 {
        return -1
    }

    min_index = 1
    max_index = 2
    i = 0
    while true {
        min_index_tmp = 2 * i + 1
        if min_index_tmp < length(data) {
            // переносим значение
            // минимального
            // индекса на min_index_tmp
            // также двигаем i на
            // min_index_tmp
            continue
        }
        break
    }

    ...

    result = data[min_index] * data[max_index]
    return result
}
```

Поиск максимального произведения



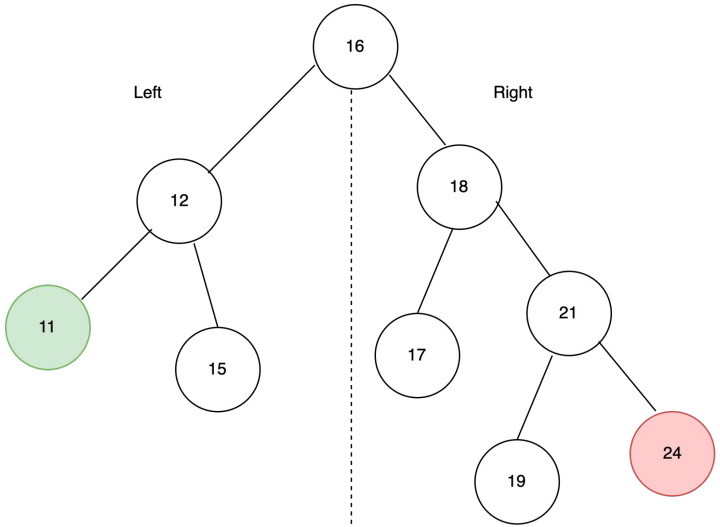

```
function maxMinMultiplication(data) {
    if length(data) < 3 {
        return -1
    }

    min_index = 1
    max_index = 2
    i = 0
    while true {
        min_index_tmp = 2 * i + 1
        if min_index_tmp < length(data) {
            min_index = min_index_tmp
            i = min_index_tmp
            continue
        }
        break
    }

    ...

    result = data[min_index] * data[max_index]
    return result
}
```

Поиск максимального произведения



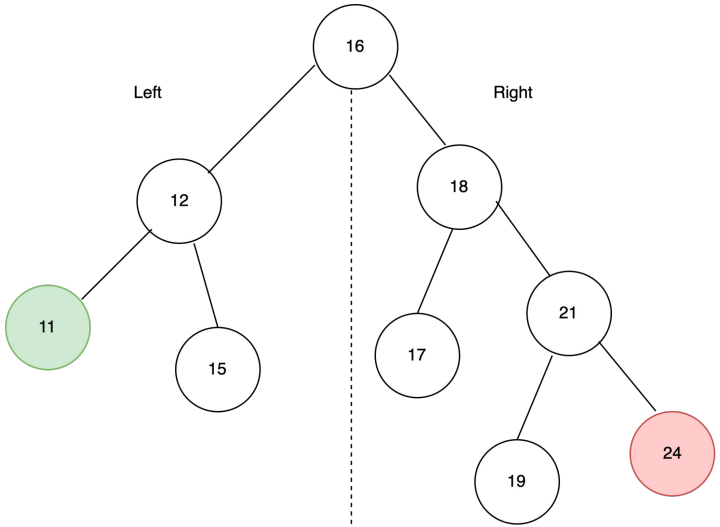

```
function maxMinMultiplication (data) {
    if length(data) < 3 {
        return -1
    }

    min_index = 1
    max_index = 2
    for i = 1; i < length(data); i = 2 * i + 1 {
        min_index = i
    }

    ...

    result = data[min_index] * data[max_index]
    return result
}
```

Поиск максимального произведения



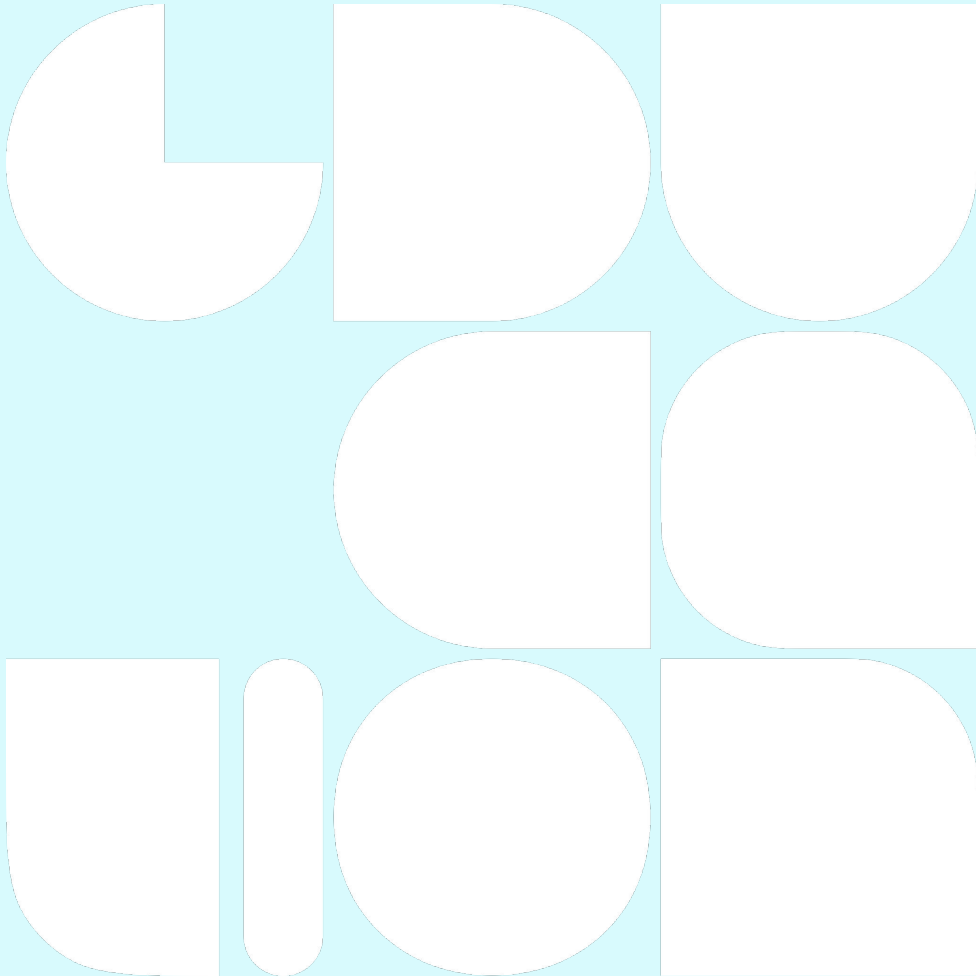

```
function maxMinMultiplication (data) {
    if length(data) < 3 {
        return -1
    }

    min_index = 1
    max_index = 2
    for i = 1; i < length(data); i = 2 * i + 1 {
        min_index = i
    }

    for i = 1; i < length(data); i = 2 * i + 2 {
        max_index = i
    }

    result = data[min_index] * data[max_index]
    return result
}
```

Сравнение двух бинарных деревьев

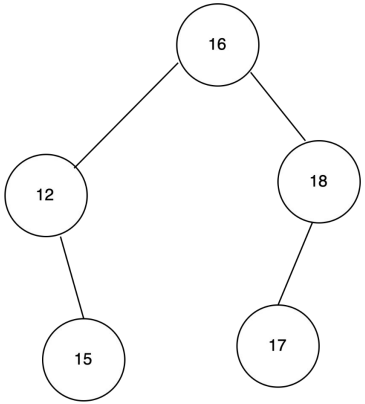


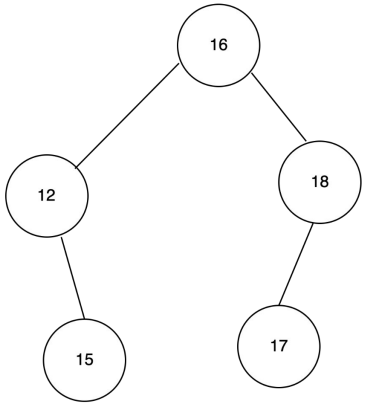
Сравнение двух деревьев

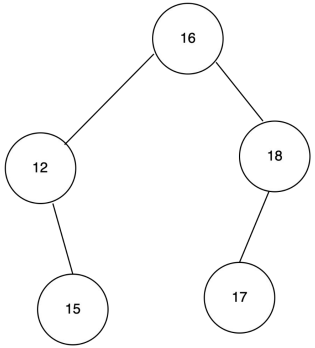
- На вход функции подается 2 бинарных дерева.
- Необходимо понять, являются ли эти два дерева одинаковыми.

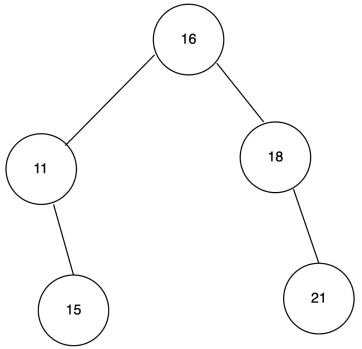
Являются ли два дерева одинаковыми

- BFS
- DFS (NLR NRL)
- Сравниваем два массива









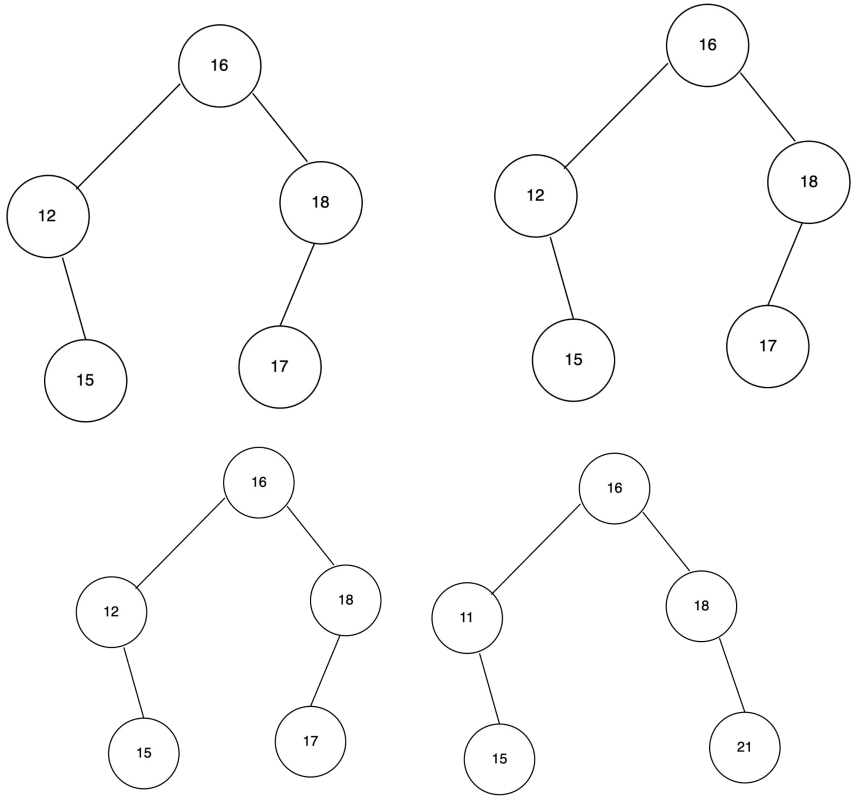
Являются ли два дерева одинаковыми

- Рекурсивный подход
- На каждом вызове сравниваем соответствующие поддеревья

```
function isSameTree(a, b) {
    if a == null && b == null {
        return true
    }

    ...

    return ?
}
```



Являются ли два дерева одинаковыми

- Если текущие поддеревья равны null, значит деревья равны
- Если только одно из поддеревьев равно null, значит на этом уровне деревья не равны

```
function isSameTree(a, b) {
    if a == null && b == null {
        return true
    }
    if a == null || b == null {
        return false
    }
    ...

    return ?
}
```

Являются ли два дерева одинаковыми

- ➔ Сравниваем текущие значения

```
function isSameTree(a, b) {
    if a == null && b == null {
        return true
    }
    if a == null || b == null {
        return false
    }
    if a.data != b.data {
        return false
    }

    return ?
}
```

Являются ли два дерева одинаковыми

- Рекурсивно сравниваем соответствующие левые и правые узлы

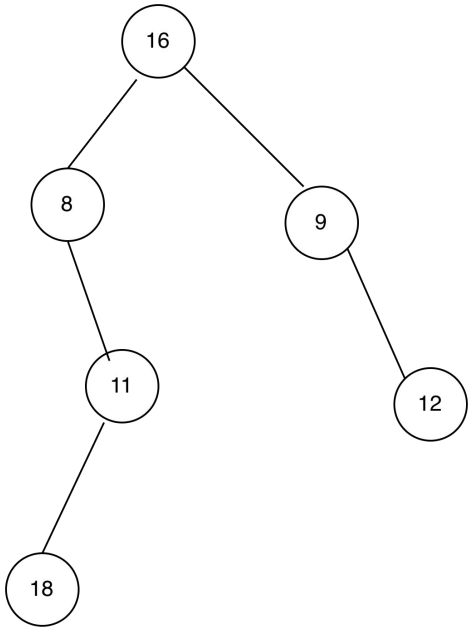
```
function isSameTree(a, b) {
    if a == null && b == null {
        return true
    }
    if a == null || b == null {
        return false
    }
    if a.data != b.data {
        return false
    }

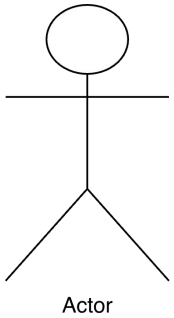
    return isSameTree(a.left, b.left) &&
     isSameTree(a.right, b.right)
}
```

Вид бинарного дерева справа

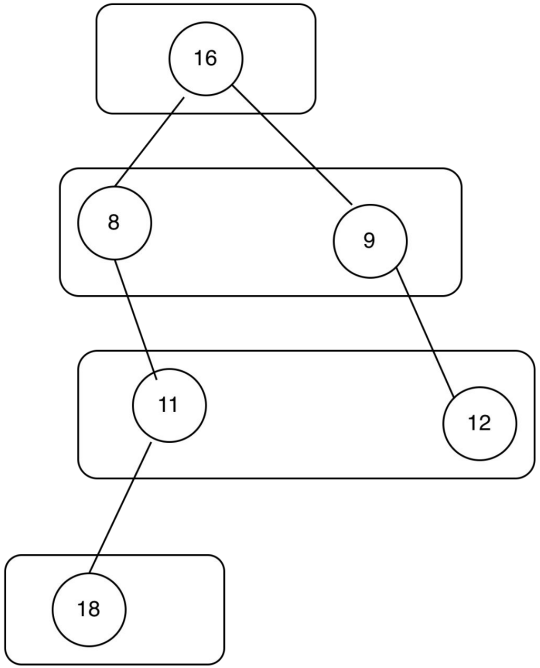
Дано бинарное дерево. Необходимо вернуть массив узлов, которые мы увидим, если будем смотреть на дерево справа.

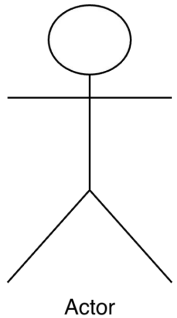
Вид бинарного дерева справа



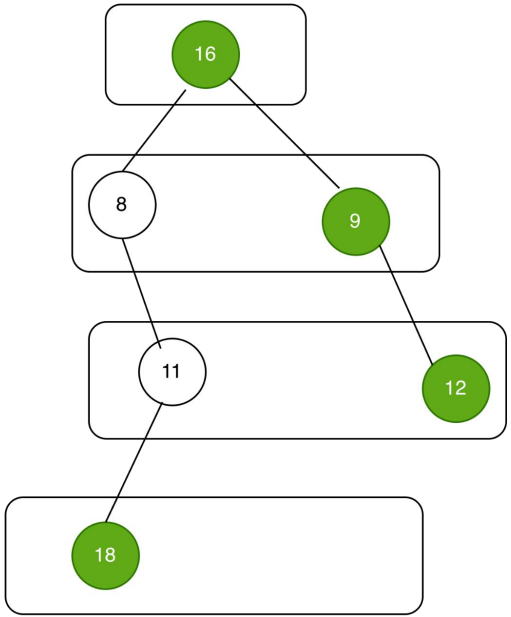


Вид бинарного дерева справа

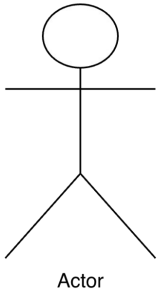




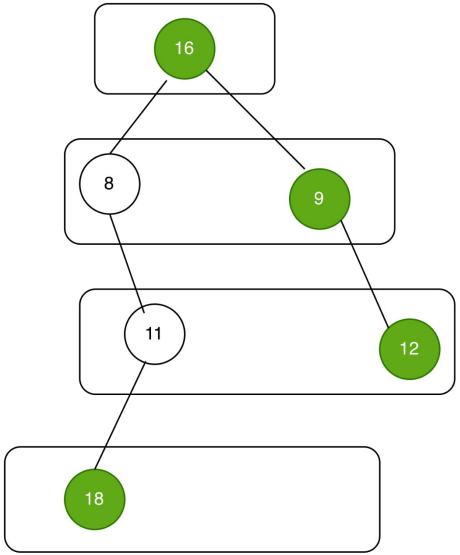
Вид бинарного дерева справа

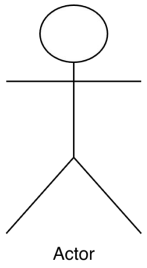


[16, 9, 12]



Вид бинарного дерева справа






```
function rightSideViewBFS(root) {
 if (root == null) {
   return []
 }

 queue = [root]
 result = []

 while (len(queue) > 0) {

   size = len(queue)
   i = 0

   while (i < size) {
     node = queue.pop()
     if (i == size - 1) {
       result.push(node.val)
     }
     if (node.left != null) {
       queue.push(node.left)
     }
     if (node.right != null) {
       queue.push(node.right)
     }

     i = i + 1
   }
 }

 return result
}
```

Спасибо за внимание!

И хорошего вечера :)

education